# Python, Perfectionnement

Darko Stankovski

19 oct. 2026

# Table des matières

**Présentation de la formation** **6**
Pré-requis . . . . . 6

**Première partie** **L'environnement de travail** **7**

**1** **L'environnement de travail** **8**
1.1 Isoler son environnement de travail . . . . . 8
1.2 Gérer des dépendances de manière standard . . . . . 10
1.3 L'approche packaging standard . . . . . 11

**Deuxième partie** **Rappels indispensables** **13**

**2** **Données et variables** **15**
2.1 Données mutables et non mutables . . . . . 15
2.2 Affectation par référence . . . . . 15
2.3 Passage de paramètres . . . . . 16
2.4 Valeurs par défaut des paramètres . . . . . 16
2.5 Visibilité locale et globale . . . . . 17
2.6 Formatage des dates . . . . . 17

**3** **Opérateurs et expressions** **18**
3.1 Opérateurs de comparaison et booléens . . . . . 18
3.2 Le Slicing . . . . . 20
3.3 Opérateur ternaire . . . . . 21
3.4 Itérer sur plusieurs séquences . . . . . 21
3.5 Notation par intension . . . . . 22
3.6 Packing et unpacking . . . . . 22

**Troisième partie** **Fonctions avancées** **24**

**4** **Notion de Closure** **25**
4.1 Généralités . . . . . 25
4.2 Modifier l'environnement d'exécution d'une closure . . . . . 25
4.3 Accéder au contexte de l'environnement d'exécution . . . . . 26

**5 Les décorateurs 27**
5.1 Structure générale des décorateurs . . . 27
5.2 Modifier le comportement d'une fonction . . . 28
5.3 Valeur en retour . . . 29
5.4 Décorateur paramétré . . . 29
5.5 Bonus : remplacer les métadonnées du wrapper . . . 29

**6 Rappel sur les générateurs 31**
6.1 Générateurs avec le mot clef yield . . . 31
6.2 Générateurs avec la syntaxe de Lists Comprehension . . . 31
6.3 D'autres générateurs, mais pourquoi ? . . . 32
6.4 Mais après tout, qu'est-ce qu'un générateur ? . . . 33
6.5 Recevoir avec `yield` . . . 33
6.6 La structure `yield from` . . . 34

**Quatrième partie Programmation Orientée Objet avancée 35**

**7 Rappels 37**
7.1 Attributs de classe et d'instance . . . 37

**8 Properties et descripteurs 38**
8.1 Les descripteurs . . . 38
8.2 Les properties . . . 39

**9 Les classes et méthodes abstraites 43**
9.1 Exemple de classe abstraite . . . 44
9.2 Spécialiser une classe abstraite . . . 44

**10 L'héritage multiple 46**
10.1 Modèle de travail . . . 46
10.2 L'ordre de résolution des méthodes . . . 48
10.3 Résolution du problème des paramètres . . . 49
10.4 Les mixins . . . 49

**11 Les context managers 51**
11.1 Présentation générale . . . 51
11.2 Un context manager reste un objet . . . 52
11.3 Idiome with as . . . 52
11.4 Paramètres de `__exit__()` . . . 53
11.5 Propagation de l'exception . . . 53

**12 Les itérateurs 54**
12.1 Exemple de base . . . 54
12.2 Vers des fonctionnalité de collections . . . 55

**13 Les métaclasses 56**
13.1 Définir dynamiquement une classe . . . 57
13.2 Créer nos propres metaclasses . . . 58

**Cinquième partie Initiation au calcul distribué 59**
Terminologie . . . 60

**14 Introduction à threading 61**
14.1 Les threads en tant que daemon . . . 62
14.2 Synchroniser les thread . . . 62
14.3 Travailler avec plusieurs threads . . . 62
14.4 Les *race conditions* . . . 63
14.5 Limiter le nombre de threads avec les sémaphores . . . 65
14.6 Spécialiser les Threads . . . 65

**15 Introduction au multiprocessing 67**
15.1 Utilisation de la mémoire partagée . . . 67

**16 Le module `concurrent.futures` 69**
16.1 Exemple . . . 69

**17 Le problème des producteurs et consommateurs 71**
17.1 Illustration . . . 71
17.2 Différents types de files . . . 72
17.3 La notion de sentinelle . . . 72
17.4 Queues, files et autres implémentations . . . 72

**18 Programmation parallèle avec Asyncio 74**
18.1 Déclaration d'une coroutine . . . 74
18.2 Rendre la main . . . 74
18.3 Créer des tâches . . . 75

**Sixième partie Initiation à la Data Science 77**

**Septième partie Environnement de travail et packaging 79**

**19 Structure de projet Python 80**
19.1 Structure proposée par Kenneth Reitz . . . 80
19.2 Structure src-based . . . 81
19.3 Références . . . 83

**20 Les outils de développement 84**
20.1 Isoler les environnements : venv . . . 84
20.2 Création d'un environnement virtuel . . . 84

20.3 Usage et activation de l'environnement virtuel . . . . . 84
20.4 Gérer les dépendances : `pip` . . . . . 85
20.5 uv, Gestionnaire de projet Python *moderne* . . . . . 86
20.6 Créer un projet . . . . . 86
20.7 Références . . . . . 89

**21 Créer un package 90**
21.1 Utiliser `setuptools` . . . . . 90
21.2 Utiliser `uv` . . . . . 90

# Présentation de la formation

Ce document est un support de formation destiné à être utilisé suite à mes formations.

Ce document est préparé pour permettre d'accéder aux informations présentées en formation et non pour être lu comme un manuel.

## Pré-requis

Cette formation est destinée à une audience ayant une maitrise des fondamentaux Python.

# Première partie

# L'environnement de travail

# 1 L'environnement de travail

Bien que vous puissiez écrire un script Python dans un fichier n'importe où dans votre système de fichiers, travailler de manière plus robuste nécessite de bien préparer son environnement de travail.

## 1.1 Isoler son environnement de travail

Un projet Python, c'est un interpréteur (sa version) et un ensemble de dépendances. Plusieurs interpréteurs peuvent cohabiter sur un même système. Les dépendances vont être spécifiques à une version d'interpréteur (installer `pytest` pour Python 3.13 ne le rends pas disponible pour 3.14). Chaque interpréteur a donc **un** environnement d'exécution. Si vous avez deux projets Python utilisant la même version d'interpréteur, elles utiliseront aussi le même environnement avec les mêmes dépendances.

Pour isoler les environnement par projet, nous utilisons les **venv**. L'utilitaire pour les créer fait parti du package standard.

Ci dessous, vous avez les grandes lignes mais consultez la documentation : https://docs.python.org/3/library/venv.html pour plus de précisions.

### 1.1.1 Création de l'environnement virtuel

L'outil s'utilise en ligne de commande et il faut commencer par créer l'environnement virtuel. La commande est de la forme :

```
1 python3.XX -m venv [ENV_DIR]
```

où :

- `python3.XX` est la version de Python utilisée pour créer le `venv` et sera la version active de cet environnement. En fonction de votre système, vous pourrez avoir besoin de déclarer le chemin absolu.
- `-m venv` est la commande de création du `venv`.
- `[ENV_DIR]` est le nom du répertoire où sera créé le `venv`. De manière générale, il est créé à la racine du projet et est nommé `.venv`.

La commande habituelle, exécutée dans le répertoire du projet, est sous cette forme (en supposant l'usage de `python3.14`) :

```
python3.14 -m venv .venv
```

### 1.1.2 Activation de l'environnement virtuel

Créer un environnement virtuel consiste à créer un répertoire. Pour pouvoir l'utiliser, il faut l'activer. La commande va dépende re votre système. Les exemples suivants considèrent que l'environnement virtuel a été créé dans un répertoire `.venv`.

Sous `sh`, `bash` ou `zsh` :

```
source .venv/bin/activate
```

Sous Windows, avec le PowerShell

```
.venv/bin/Activate.ps1
```

Sous Windows, avec le `cmd.exe`

```
.venv\Scripts\activate.bat
```

### 1.1.3 Désactiver un environnement virtuel

Si vous avez besoin de sortir de l'environnement virtuel, c'est à dire de le désactiver dans le terminal, l'instruction est :

```
deactivate
```

### 1.1.4 Supprimer un environnement virtuel

Supprimer un environnement virtuel consiste à supprimer le répertoire qui le contient.

### 1.1.5 Les environnements virtuels et les gestionnaires de version

Le répertoire d'un environnement virtuel **n'est pas sous contrôle de version**. Ces environnement sont volumineux et peuvent contenir des packages spécifiques à votre système (OS ou architecture). Les environnements virtuels se reconstruisent.

### 1.1.6 Les environnements virtuels et les IDEs

Les principaux IDEs (PyCharm et Visual Studio Code) on des outils plus *graphiques* de gestion des environnements virtuels. Il ne s'agit cependant que d'une interface sur les instructions présentées ici.

## 1.2 Gérer des dépendances de manière standard

Les dépendances se gèrent de manière standard avec `pip`.

### 1.2.1 Installer une dépendance

Pour installer une dépendance :

```
pip install pytest
```

Vous pouvez installer plusieurs dépendances en une instruction

```
pip install pytest pylint ruff
```

### 1.2.2 Mettre à jour une dépendance

```
pip install --upgrade pytest
```

Pour information, `pip` se met à jour par lui même :

```
pip install --upgrade pip
```

### 1.2.3 Supprimer une dépendance

```
pip uninstall pytest
```

### 1.2.4 Lister les dépendances

```
pip list
```

### 1.2.5 Gérer les dépendances d'un projet, approche historique

Historiquement, les dépendances d'un projet sont déclarées dans un fichier `requirements.txt`, ce qui facilite leur installation. Ce fichier doit suivre une certaine syntaxe. Le plus simple est de le générer, à l'aide de l'instruction :

```
pip freeze
```

Cette instruction affichera dans le terminal les bibliothèques installées avec la syntaxe suivante :

```
flake8==5.0.4
ipython==8.11.0
pylint==2.15.5
pytest==7.0.1
```

Pour créer le fichier, il suffit de rediriger la sortie standard vers le fichier :

```
pip freeze > requirements.txt
```

Pour installer le contenu du fichier :

```
pip install -r requirements.txt
```

Il est également possible d'utiliser ce fichier pour mettre à jour des dépendances :

```
pip install --upgrade -r requirements.txt
```

## 1.3 L'approche packaging standard

Un programme Python, dans sa forme la plus simple, est une arborescence de fichiers. Python prévoit également la possibilité de *packager* le code pour en faite un *package*. C'est exactement le type de fichiers que vous installez avec `pip instal`. Mais créer un projet destiné au packaging a aussi un intérêt dans le cadre du développement.

Un projet destiné au packaging a une structure particulière (*src-based layout*) et est décrit dans un fichier `pyproject.toml`.

Il y a plusieurs intérêts pour le développement.

### 1.3.1 Installation du projet en mode éditable

```
pip install -e .
```

Le projet actuel n'est pas installé, un lien est créé dans la bibliothèque des packages vers l'arborescence. Ceci permet de ne plus avoir de problème de chemin vers les ressources.

### 1.3.2 Une gestion centralisée des dépendances

Lorsqu'un projet est destiné au packaging, la création d'un package ne regarde pas le fichier `requirements.txt`. Les dépendances sont décrites dans le `pyproject.toml`.

Depuis `pip 26.2`, il est possible d'utiliser le fichier `pyproject.toml` et se passer du `requirements.txt` :

```
pip install . --only-deps
```

En général, dans un `pyproject.toml`, les dépendances ne sont que les dépendances *de prod*. Les dépendances de développement sont un *groupe* habituellement nommé `dev` et que l'on peut installer par :

```
pip install --group dev
```

# Deuxième partie

# Rappels indispensables

Le présent chapitre rappelle des notions de base pas toujours connues ou maitrisées mais pourtant fondamentales pour bien comprendre certaines parties de la formation perfectionnement.

# 2 Données et variables

## 2.1 Données mutables et non mutables

Python possède la notion de données mutables et non mutables. Les premières sont modifiables, pas les secondes.

Parmi les types de base, les chaines de caractères (`str`) et les N-Uplets (`tuple`) sont immuables. Il ne sera donc pas possible d'ajouter, supprimer ou remplacer un élément. Les expressions ci-contre lèvent une exception.

```
king = "Arthur"
king[0] = "P"
del king[0]
```

Parmi les types communs, nous avons aussi par exemple les dates avec `datetime`, les durées avec `timedelta` ou les chemins de type `Path` qui sont immuables.

## 2.2 Affectation par référence

En Python, les variables contiennent une référence vers une données. Affecter une variable à un autre copie la variable, c'est à dire la référence vers la données. Il n'y a pas de duplication de donnée. Ceci permet une économie de mémoire.

Ceci peut être vérifié en utilisant la fonction `id()` sur la variable (donc sur la donnée). De même, l'opérateur `is` est un comparateur d'identité.

```
king = "Arthur"
apprentice = king

print(f"{king=}, {apprentice=}")                    ①
print(f"{id(king)=}, {id(apprentice)=}")            ②
print(f"{king is apprentice=}")                     ③
```

① Les deux variables contiennent la même chaine de caractères
② L'id référencé par les deux variables est le même
③ Affiche `True`car les deux chaines sont les mêmes

Avec les types mutables, la donnée unique sera modifiée. Un mauvais usage provoquera un effet de bord mais un bon usage permet également d'éviter une recherche pour retrouver une donnée à modifier.

```
kingdom = ["Arthur", "Merlin", "Lancelot"]
camelot = kingdom

camelot.append("Robin")
print(f"{kingdom=}")
```

## 2.3 Passage de paramètres

Le passage de paramètres se fait également par référence. Si vous passez une donnée mutable en paramètre d'une fonction, celle-ci pourra être modifiée par la fonction.

```
def add_to_kingdom(knight:str, kingdom):
    kingdom.append(knight)

camelot = []
add_to_kingdom("Lancelot", camelot)
print(f"{camelot=}")                                    ①
```

① La liste `camelot` a bien un élément en plus

Il y a un idiome en Python : si une données passée en paramètre à une fonction qui doit la modifier est retournée par la fonction, il s'agit d'une copie. Si la fonction ne la retourne pas, alors la donnée originale a été modifiée. Vous en avez une illustration avec la méthode `.sort()` du type `list` et la fonction `sorted()`.

## 2.4 Valeurs par défaut des paramètres

Les paramètres par défaut sont initialisés qu'une seule fois, lors d'appels successifs, ce sont les même références qui sont utilisées. Ceci peut provoquer une mauvaise surprise avec les types mutables comme les listes.

```
def add_to_kingdom(knight:str, kingdom=[]):
    kingdom.append(knight)
    return kingdom

print(f'{add_to_kingdom("Lancelot")=}')                 ①
print(f'{add_to_kingdom("Robin")=}')                    ②
```

① La liste retournée contient `['Lancelot']`
② La liste retournée contient `['Lancelot', 'Robin']`

## 2.5 Visibilité locale et globale

Comme pour tous les langages, les variables ont une visibilité locale au bloc qui les déclare. Elles sont également visibles aux sous blocs comme les fonctions en lecture seule (par défaut).

En Python, les blocs des structures de contrôle (`for`, `while` et `if`) ainsi que les blocs liés aux exceptions sont considérés comme le bloc *courant*.

On parle d'une variable globale quand une variable est déclarée au niveau du module Python. On parle de variable et de visibilité locale lorsqu'une variable est déclarée au niveau d'un *bloc*.

## 2.6 Formatage des dates

Les manières moderne pour formater des dates est l'utilisation de la méthode `.format()` des chaines de caractères ou les f-string. Dans les deux cas, il n'est pas nécessaire de faire appel à la méthode `.strftime()` du type `datetime`. Le formatage sera pris en charge par le descripteur de formatage.

```
1 from datetime import datetime
2
3 xmas2022 = datetime(2022, 12, 25)
4 print("Xmas is on {:%B %d %Y}".format(xmas2022))  # 'December 25 2022'
```

# 3 Opérateurs et expressions

## 3.1 Opérateurs de comparaison et booléens

### 3.1.1 Les booléens

Le type `bool` ne contient que deux valeurs : `True` et `False`.

Les opérateurs booléens ne se limitent pas à l'usage de types booléens. D'autres types peuvent être utilisés et certaines de leurs valeurs seront considérées comme fausses (on parle quelquefois de *falsy* pour éviter la confusion). Ainsi, les collections vides, les nombres nul (`0`, `0.0`…) ou le type `None` sont considérés comme falsy.

| Type | FAUX |
|---|---|
| Bool | False |
| int | 0 |
| float | 0.0 |
| string | “ ” |
| tuple | () |
| list | [] |
| dict | {} |
| None Type | None |

On peut donc directement tester les valeurs sans les convertir en type `bool`.

```
knights = []

if knights:
    print("Go for the Grail")
else:
    print("need to hire knights…")
```

### 3.1.2 Les opérateurs logiques

Les opérateurs logique sont, par ordre de priorité, `or`, `and` et `not`

La négation `not` retourne une donnée de type `bool`. L'opérateur de conjonction `and` et l'opérateur de disjonction `or` nécessitent deux expressions à évaluer (soit `x` et `y` ces expressions)

et sont considérés comme paresseux. Cela signifie que l'expression de droite ne sera évaluée que si nécessaire. De plus, ces opérateurs ne retournent pas de valeur de type booléenne. Ceci s'explique par leur comportement qui est :

La conjonction `x and y` retournera `y` si `x` sinon `x`.

La disjonction `x or y` retournera `x` si `x` sinon `y`.

```
army = []
heroes = ["lancelot", "Perceval"]

print(f"{heroes or army=}")                                    ①
print(f"{army or heroes=}")

print(f"{army and heroes=}")                                   ②
```

① Le retour affiché de cet opérateur sera `['lancelot', 'Perceval']`
② Le retour affiché de cet opérateur sera `[]`

### 3.1.3 La structure `for else`

Les blocs des boucles (`for` ou `while`) peuvent être suivis d'un bloc `else`. Ce bloc sera évalué si la sortie de la boucle s'est faite autrement que par un break, autrement dit si la sortie de la boucle a eu lieu *normalement* après avoir parcouru toutes les itérations.

C'est une structure qui se justifie principalement avec le bloc for et évite l'usage d'un *flag*. Le choix du mot clef peut prêter à confusion mais il correspond à l'intention qui est la recherche d'un élément où, si il n'a pas été trouvé, nous avons une action à réaliser. La lecture doit être « Je recherche un élément, je parcours tous les éléments de ma séquence. **Si** je l'ai trouvé, **break**, **sinon**…).

```
camelot = ['Arthur', 'Merlin', 'Lancelot', 'Robin']
wizards = ['Merlin', 'Tim']

for peon in camelot:
    if peon in wizards:
        print("The kingdom has a wizard")

else:
    "Sir, you need to find a wizard"
```

Nous pouvons aussi utiliser un bloc `else` avec les exceptions. La clause `else` suit tous les blocs `except` et permet d'exécuter du code si aucune exception n'a été levée. Si vous l'utilisez, la clause `finally` doit toujours être la dernière déclarée.

## 3.2 Le Slicing

Le slicing est un outil très utile en Python. Il permet d'accéder à des *slices* de séquences ou aux éléments en fonction d'un pas. Mais il peut aussi être utilisé d'autres manières

### 3.2.1 Affectations

La notation par slice peut permettre de remplacer un nombre d'éléments différent dans une liste.

```
numbers = ["one", "two", "three", "four", "five", "six", "seven"]
letters = ["a", "b", "c"]

numbers[1:3] = letters
print(numbers)                                                    ①

numbers = ["one", "two", "three", "four", "five", "six", "seven"]
numbers[1:6] = letters
print(numbers)                                                    ②
```

① Affichera `['one', 'a', 'b', 'c', 'four', 'five', 'six', 'seven']`
② Affichera `['one', 'a', 'b', 'c', 'seven']`

### 3.2.2 Suppression d'éléments

L'instruction `del` sur un élément d'une liste supprime l'élément. L'instruction `del` peut être utilisée avec les slices pour supprimer plusieurs éléments

```
numbers = ["one", "two", "three", "four", "five", "six", "seven"]
del numbers[2]
print(numbers)                                                    ①

numbers = ["one", "two", "three", "four", "five", "six", "seven"]
del numbers[2:5]
print(numbers)                                                    ②

numbers = ["one", "two", "three", "four", "five", "six", "seven"]
del numbers[::2]
print(numbers)                                                    ③
```

① Affichera `['one', 'two', 'four', 'five', 'six', 'seven']`
② Affichera `['one', 'two', 'six', 'seven']`
③ Affichera `['two', 'four', 'six']`

### 3.2.3 Object Slice

Les slices sont aussi un objet en Python. Un objet `slice(start, stop, step)` peut être passé à la place d'entiers.

```
numbers = ["one", "two", "three", "four", "five", "six", "seven"]
my_slice = slice(2, 4)                                              ①
```

① `my_slice` contient `['three', 'four']`

## 3.3 Opérateur ternaire

Python propose la syntaxe suivante pour l'opérateur ternaire :

```
valeur_si_vrai if condition else valeur_si_faux
```

L'opérateur ternaire est à utiliser pour évaluer une valeur en fonction d'une condition. Notez que l'opérateur ternaire retourne une valeur, les parties de gauche (si vrai) ou droite (si faux) doivent donc retourner une valeur.

## 3.4 Itérer sur plusieurs séquences

La fonction `zip()` permet d'itérer sur plusieurs séquences. L'objet retourné est un itérateur qui retourne un tuple des éléments des séquences itérées.

Un usage simple peut être le suivant :

```
camelot = ["Arthur", "Merlin",
           ("Lancelot", "Robin", "Perceval", "Karadoc")]
roles = ["king", "wizard", "knights"]

for role, peon in zip(roles, camelot):
    print(role, peon)
```

Si les séquences n'ont pas le même nombre d'éléments, l'itération s'arrêtera à la fin de la plus courte.

## 3.5 Notation par intension

Les Comprehension lists sont un outil très pratique en Python pour transformer ou filtrer des listes.

Mais cette notation n'est pas limitée aux listes. Nous pouvons par exemple l'utiliser avec des dictionnaires.

Nous pouvons d'ailleurs combiner cette propriété avec la fonction `zip()` :

```
1 print({role:peon for role, peon in zip(roles, camelot)})
```

## 3.6 Packing et unpacking

L'unpacking (ou *déballage* en français) permet d'extraire automatiquement des éléments d'une séquence. Vous l'avez certainement déjà vu avec l'affectation d'éléments d'une liste ou d'un tuple à des variables.

```
1 hour, minutes = divmod(75, 60)
```

Et vous l'avez certainement déjà utilisé lors des itération sur un `enumerate()`.

Dans ce cas, l'unpacking est *automatique*. Il est cependant possible de le *forcer* à l'aide du *Star Operator* `*`.

Attention, cet opérateur ne s'utilise que dans certains contextes. Et en fonction du contexte, c'est un opérateur de déballage ou d'emballage (packing/unpacking).

### 3.6.1 Opérateur d'unpacking

Vous pouvez l'utiliser pour déballer une séquence dans un contexte qui attends la séquence ou les éléments de la séquence comme les paramètres d'une fonction.

```
1 print("Il est {:02}h{:02}".format(*divmod(75, 60)))
```

Dans cet exemple, sur les heures obtenus par divmod(), vous n'avez pas besoin de passer par des variables transitoires.

### 3.6.2 Opérateur de packing

Le star operator peut aussi être utilisé pour *emballer* des éléments. Il peut être utile lorsqu'on utilise le *déballage automatique* lors de l'affectation d'une séquence à des variables. Il permet ainsi de récupérer les données *excédentaires*.

```
1 king, wizard, *others = camelot
```

### 3.6.3 Inversion du contenu de variables

Vous connaissez certainement l'idiome `a, b = b, a` qui permet d'inverser le contenu de deux variables.

Dans la pratique, il met en œuvre l'unpacking. En effet, à droite de l'affectation, vous avez la création d'un tuple de deux éléments. En affectant son contenu à deux variables, vous déballez dans la même action le tuple.

# Troisième partie

# Fonctions avancées

# 4 Notion de Closure

## 4.1 Généralités

De manière générale, une fonction retient le contexte dans lequel elle est créée. Une fonction retournée par une fonction retient donc de la même manière le contexte dans lequel elle a été créée :

```
def outer(param):
    value = param
    def inner(modif):
        return value * modif
```

Va s'utiliser :

```
>>> my_answer = outer(42)
>>> my_answer(10)
420
```

## 4.2 Modifier l'environnement d'exécution d'une closure

Comme pour toute variable, les variables définies dans la fonction externe ne sont visibles qu'en lecture seule pour la fonction interne. Il faut utiliser la déclaration `nonlocal` pour indiquer à l'interpréteur que l'on travail dans l'espace de la fonction englobante.

```
def outer(name):
    count = 0
    def inner():
        print(f"inner func for {name}")
        nonlocal count
        count += 1

    return inner
```

## 4.3 Accéder au contexte de l'environnement d'exécution

Le contexte d'exécution défini par la fonction externe n'est pas accessibles dans l'environnement qui a récupéré la fonction interne. L'astuce consiste à définir des fonctions attachées à la fonction retournée.

```
def outer(name):
    count = 0

    def inner():
        print(f"inner func for {name}")
        nonlocal count
        count += 1

    def inner_get_count():
        return count

    inner.get_count = inner_get_count

    return inner
```

Nous pouvons alors utiliser :

```
>>> click = outer('demo')
>>> click()
>>> click()
>>> click.get_count()
2
```

# 5 Les décorateurs

Un décorateur est un outil récurent dans le langage Python qui sert à modifier le comportement d'une fonction ou d'une méthode. Selon les domaines d'application, ils sont plus ou moins fréquents.

Un décorateur est déclaré avant la définition d'une fonction (sur la ligne précédente) par `@`.

```
@decorator
def func():
    pass
```

Python a un nombre limité de décorateurs built-in qui sont limités à l'Objet. Il s'agit de `@staticmethod`, `@classmethod` et `@property`.

Les décorateur sont des éléments qui vont nous permettre d'aborder la notion de la métaprogrammation. La métaprogrammation désigne l'écriture de programmes qui manipulent des données décrivant elles-mêmes des programmes…

Bref, les décorateurs sont des composants qui vont nous permettre de modifier le comportement de fonctions ou de classes en exécutant du code avant et/ou après l'exécution normal de cette fonction. En soi, **ils sont une mise en œuvre des Closures**.

Nous allons dans cette partie voir comment déclarer son propre décorateur.

## 5.1 Structure générale des décorateurs

Un décorateur est une fonction qui prend en paramètre une fonction et retourne une fonction. La déclaration de base est donc :

```
def deco_func(func):
    return func
```

L'usage pourrait être :

```
def say_hello():
    print('Hello World')

deco_func(say_hello)()
```

Mais c'est le strict nécessaire pour utiliser `deco_func()` comme décorateur :

```

@deco_func
def say_hello():
    print('Hello World')
```

## 5.2 Modifier le comportement d'une fonction

Pour exécuter quelque chose avant et/ou après la fonction décorée, cette fonction est empaquetée dans une fonction interne.

```
def sandwich(content):
    def wrapper():
        print("/TTTTTTTTT\\")
        content()
        print("\\_________/")

    return wrapper
```

Nous allons pouvoir décorer une fonction

```
@sandwich
def parisien():
    print("Jambon")
    print("Beurre")
```

L'appel de la fonction `parisien()` exécutera en réalité `wrapper()`. La fonction `parisien()` sera exécutée en tant que `content()` lors de cet appel.

Les décorateurs ne sont qu'une implémentation *pythonesque* (ou *pythonique*) du motif de conception *décorateur*. ## Passage d'arguments

L'usage de variadics permet une cohésion faible entre le décorateur et la fonction à décorer.

```
def sandwich(content):
    def wrapper(*args, **kargs):
        print("/TTTTTTTTT\\")
        content(*args, **kargs)
        print("\\_________/")

    return wrapper
```

## 5.3 Valeur en retour

Lorsque la fonction décorée est dans le contexte d'une fonction interne, celle-ci doit communiquer le retour.

```
def sandwich(content):
    def wrapper(*args, **kargs):
        print("/TTTTTTTTTT\\")
        value = content(*args, **kargs)
        print("\\__________/")

        return value

    return wrapper
```

## 5.4 Décorateur paramétré

Conserver la syntaxe simple avec le décorateur paramétré

```
def name(_func=None, *, key1=value1, key2=value2, ...):
    def decorator_name(func):
        ...  # La fonction wrapper

    if _func is None:
        return decorator_name
    else:
        return decorator_name(_func)
```

## 5.5 Bonus : remplacer les métadonnées du wrapper

Dans le cas de l'utilisation d'un wrapper, l'introspection de la fonction sera *faussé*. Nous n'avons plus la fonction d'origine mais la fonction wrapper. Faites le test sur la base du décorateur `sandwich`, vous devriez avoir quelque chose de similaire à :

```
>>> parisien.__name__
'wrapper'
```

Pour préserver les informations de la fonction d'origine :

```
import functools

def sandwich(content):
    @functools.wraps(content)
    def wrapper():
        print("/TTTTTTTTTT\\")
        content()
        print("\\__________/")

    return wrapper
```

# 6 Rappel sur les générateurs

Un générateur est un objet particulier en Python. Vous avez dû voir deux manières *de base* pour les créer.

## 6.1 Générateurs avec le mot clef yield

Si une fonction utilise le mot clef `yield`, elle devient un générateur.

```
camelot = ["Arthur", "Merlin", "Lancelot",
           "Robin", "Perceval", "Karadoc"]

def upperise(sequence):
    for item in sequence:
        yield item.upper()

print(upperise(camelot))  # <generator object upperise at 0x113f81c40>
```

Vous pouvez utiliser la structure `for` pour itérer sur les éléments fournis par le générateur.

```
for element in upperise(camelot):
    print(element)
```

## 6.2 Générateurs avec la syntaxe de Lists Comprehension

Si au lieu des crochets des Lists Comprehension, vous utilisez des parenthèses (ou pas selon le contexte), vous ne créez pas une liste mais un générateur.

```
print((element.upper() for element in camelot))
```

Dans cet exemple avec la fonction `print()`, les parenthèses ne sont pas nécessaires. Elles ont été conservées pour mettre en évidence la structure du générateur.

## 6.3 D'autres générateurs, mais pourquoi ?

Python propose quelques générateurs comme la fonction `range()`. Sous Python 2, cette fonction retournait une liste. La fonction `xrange()` retournait un générateur et correspond à la fonction `range()` de Python 3. Si vous souhaitez obtenir une liste d'entiers, il suffit de faire `list(range())` ce qui est généralement à éviter.

Si Python gère assez bien la mémoire, il y a quelques problèmes avec les listes d'entiers et de réels qui ne sont pas forcément libérées avant la fin du programme.

Vous pouvez vérifier ceci avec le code suivant. Celui-ci nécessite `psutil` que vous installerez avec `pip`.

```
import os
import psutil

process = psutil.Process(os.getpid())

while True:
    l = list(range(1 * 1000 * 1000))
    print(f"Memory usage: {process.memory_info()[0] / float(2 ** 20)} MB")
```

L'affichage en illustration sera le suivant :

```
Memory usage: 71.80859375 MB
Memory usage: 81.52734375 MB
Memory usage: 81.52734375 MB
Memory usage: 82.51171875 MB
Memory usage: 82.51171875 MB
…
Memory usage: 108.10546875 MB
Memory usage: 109.08984375 MB
Memory usage: 109.08984375 MB
Memory usage: 110.07421875 MB
Memory usage: 110.44921875 MB
Memory usage: 110.44921875 MB
```

Les différentes versions de Python n'ont pas le même comportement avec ce programme mais le résultat sera toujours une augmentation de la mémoire utilisée. Dans le cas présent, elle se limite à une certaine taille.

De manière générale, favorisez les générateurs pour éviter ce défaut.

## 6.4 Mais après tout, qu'est-ce qu'un générateur ?

Un générateur est un objet spécial qui permet de manipuler un *itérateur* sur la fonction. Nous pouvons alors parcourir les valeurs produites par appel à la fonction builtin `next()` ou par appel à la méthode spéciale `__next__()`.

Dans la pratique, la structure `for` appelle à chaque itération cette fonction `__next__()` du générateur.

Lorsque vous créez un générateur avec le mot-clef `yield`, le générateur arrête l'exécution de la fonction à ce mot clef et attends l'appel à la fonction `__next__()`. Elle fournit alors la valeur suivant le mot clef et reprends jusqu'au `yield` et attends à nouveau.

Nous verrons comment créer nos propres itérateurs et itérables dans la partie sur la programmation orientée objet.

## 6.5 Recevoir avec `yield`

Le mot-clef `yield` peut aussi être utilisé par un générateur pour recevoir des données qui lui seront envoyées par la méthode `.send()`. La syntaxe que vous aurez dans le générateur est de type `x = yield`.

Un exemple d'illustration d'un générateur recevant et produisant une valeur est le suivant :

```
1  def test_gen():
2      while True:
3          x = yield                    ①
4          yield x * 2                  ②
5
6  mygen = test_gen()
7  next(mygen)                          ③
8  print(mygen.send(10))                ④
9  next(mygen)                          ⑤
10 print(mygen.send(5))                 ④
```

① Cette instruction permet au générateur de recevoir une valeur.
② Suivi du renvois de la valeur.
③ Le premier `next()` est indispensable pour *faire avancer* le générateur au premier `yield`. À défaut, vous aurez une erreur : `TypeError: can't send non-None value to a just-started generator`.
④ l'envoi de valeur par `.send()` va provoquer le calcul et le retour du résultat
⑤ Il est nécessaire d'avancer le générateur.

## 6.6 La structure `yield from`

Ajoutée en Python 3.3, la structure `yield from` permet de simplifier l'appel d'un *sous-générateur*.

Ainsi, plutôt que d'écrire :

```
1 def demo_yield():
2     for item in range(10):
3         yield item
```

On écrit :

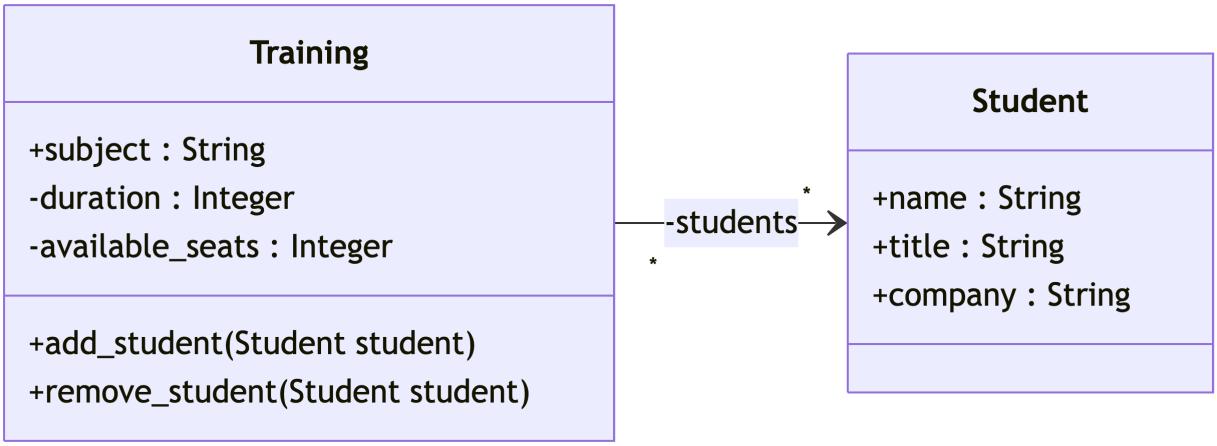
```
1 def demo_yield_from():
2     yield from range(10)
```

Notez que tant que l'action `yield from` s'exécute, le générateur `demo_yield_from` est **remplacé** par le générateur `range`.

## Quatrième partie

# Programmation Orientée Objet avancée

Pour cette partie, le point de départ du modèle récurent pour illustrer les concepts sera un modèle simplifié de gestion de formation.



# 7 Rappels

## 7.1 Attributs de classe et d'instance

Lorsqu'une variable est déclarée au niveau d'une classe, il s'agit d'un attribut de classe. Celle-ci est partagée par toutes les instances. Un attribut d'instance est communément initialisé dans le *constructeur* (en fait, l'initialiseur) `__init__()`.

```
1  class Kingdom:
2      count = 0
3      def __init__(self, name):
4          self.name = name
5          Kingdom.count += 1                                    ①
6
7  camelot = Kingdom("Camelot")
8  print(f"{camelot.name=}")
9  print(f"{camelot.count=}")
10
11 winterfell = Kingdom("Winterfell")
12 print(f"{winterfell.name=}")
13 print(f"{camelot.count=}")                                    ②
14 print(f"{Kingdom.count=}")                                    ③
```

① Un attribut de classe est accédé préfixé par le nom de la classe et non `self`.
② Pour chaque instance, la valeur de `.count` sera la même.
③ La bonne manière d'accéder à l'attribut de classe est de le préfixer par le nom de la classe.

Attention cependant, en Python, un attribut d'instance peut-être créé dynamiquement. Si la syntaxe `nom_instance.attribut_classe` permet d'accéder à l'attribut de classe, une affectation créera un attribut d'instance su même nom masquant l'attribut de classe.

# 8 Properties et descripteurs

## 8.1 Les descripteurs

Un descripteur est un objet définissant les méthodes `__get__()`, `__set__()` ou `__delete__()` (au moins une). Ils sont la base de beaucoup de fonctionnalités en Python tel que les fonctions, les méthodes, les properties, les méthodes de classe, les méthodes statiques et les références vers les super classes.

### 8.1.1 Exemple de base

```
class Answer:
    def __get__(self, obj, objtype=None):
        return 42
```

Ce descripteur doit être utilisé comme attribut de classe :

```
class Universe:
    meaning_of_life = Answer()
```

Ce qui permet un usage :

```
u = Universe()
u.meaning_of_life
```

### 8.1.2 Accéder aux attributs

Les descripteurs sont utilisés pour accéder aux attributs.

```
class TurtleAccess:
    def __get__(self, obj, objtype=None):
        return obj._turtle_name

    def __set__(self, obj, value):
        obj._turtle_name = value.title()

```

```
class DiscWorld:
    turtle_name = TurtleAccess()                                    ①

    def __init__(self, turtle_name:str):
        self.turtle_name = turtle_name                              ②

discworld = DiscWorld("a'tuin")
print(discworld.turtle_name)                                        ③
```

① Création d'une instance du descripteur `TurtleAccess`
② Appel au `__set__()` du descripteur
③ Appel au `__get__()` du descripteur et affichage de la chaine `"A'Tuin"`.

### 8.1.3 Plus généralement

Les descripteurs sont des objets, leur définition peut donc avoir un constructeur et des méthodes spécifiques.

## 8.2 Les properties

### 8.2.1 Principe général

Une `property` est un objet créé par la fonction built-in `property()`. La signature de cette fonction est :

```
property(fget=None, fset=None, fdel=None, doc=None)
```

Appeler `property()` est une manière succincte de construire un descripteur de données en fournissant :

— Une fonction `fget` pour les accès `object.property`
— Une fonction `fset` pour les accès `object.property = value`
— Une fonction `fdel` pour les expressions `del object.property`

### 8.2.2 Exemple complet

Soit la classe simplifiée :

```python
class Training:
    def __init__(self, title: str, max_seats: int|str = 10):
        self.title = title.upper()
        self.max_seats = int(max_seats)
        self._students = []

    def add_student(self, student: Student):
        self._students.append(student)
```

Nous pouvons définir les fonctions :

```python
def get_students(self):
    return self._students.copy()

def set_students(self, students: list):
    self._students[:] = students[:]

def del_students(self):
    self._students.clear()
```

et donc ajouter dans la classe `Training` :

```python
class Training:
    ...

    students = property(fget=get_students, fset=set_students,
                        fdel=del_students)
```

Mais chaque paramètre est optionnel donc vous pouvez ne déclarer que ceux qui ont un sens :

```python
class Training:
    ...

    students = property(fget=get_students)
```

Modifier ou supprimer *l'attribut* `students` entrainera un `AttributError`. Ceci permet d'avoir un attribut en lecture seule :

```python
python_t = Training("python", 5)
python_t.add_student(Student("Kevin", "Mitnick"))
python_t.add_student(Student("Elliot", "Alderson"))

print(python_t.students)                                    ①
python_t.students = []                                      ②
```

① Affichage dans la console de `[Student Kevin, Mitnick, Student Elliot, Alderson]`
② Lève une exception `AttributeError: can't set attribute 'students'`

### 8.2.3 Déclarer une property par décorateur

La déclaration de properties est simplifiée par le décorateur `property`. Au lieu de définir des fonctions et la variable de classe `property()`, vous définissez l'accesseur comme une méthode standard mais décorée avec `@property`.

```
@property
def students(self):
    return self._students.copy()
```

Python créera alors l'attribut de classe `students` avec cette méthode en tant que `fget`.

Si vous souhaitez ajouter également les fonctions `fset` et/ou `fdel`, vous déclarez les méthodes correspondantes mais il faut utiliser un décorateur qui lie ces fonctions à la property créée par l'accesseur. Pour cela, cette property propose 2 fonctions à utiliser en décorateur :

```
@students.setter
def students(self, students: list):
    self._students[:] = students[:]

@students.deleter
def students(self):
    self._students.clear()
```

### 8.2.4 Limite de la syntaxe par décorateurs

Avec la fonction `property`, vous pouvez créer des properties avec toutes les combinatoires écriture, lecture et suppression. Avec le décorateur, la property sera créé en lecture. Vous ne pouvez donc pas utiliser le décorateur si vous ne voulez pas accéder à la property.

### 8.2.5 Properties et encapsulation

L'exemple d'usage des properties pour contrôler l'accès à un attribut réel de l'objet est la première étape et l'explication la plus simple. Mais les properties permettent de réellement mettre en œuvre la notion d'encapsulation consistant à *masquer l'implémentation* : elles permettent d'exposer un état décorrélé de l'implémentation.

Prenons l'exemple du nombre de places de la formation. Elle est créée avec le nombre de places total mais nous pouvons vouloir savoir combien de places il y a de disponibles. Cette nformation est un état donc un attribut mais qui dépends de deux autres états. Il peut donc être évalué à chaque demande par une property :

```python
@property
def available_seats(self):
    return self.max_seats - len(self._students)
```

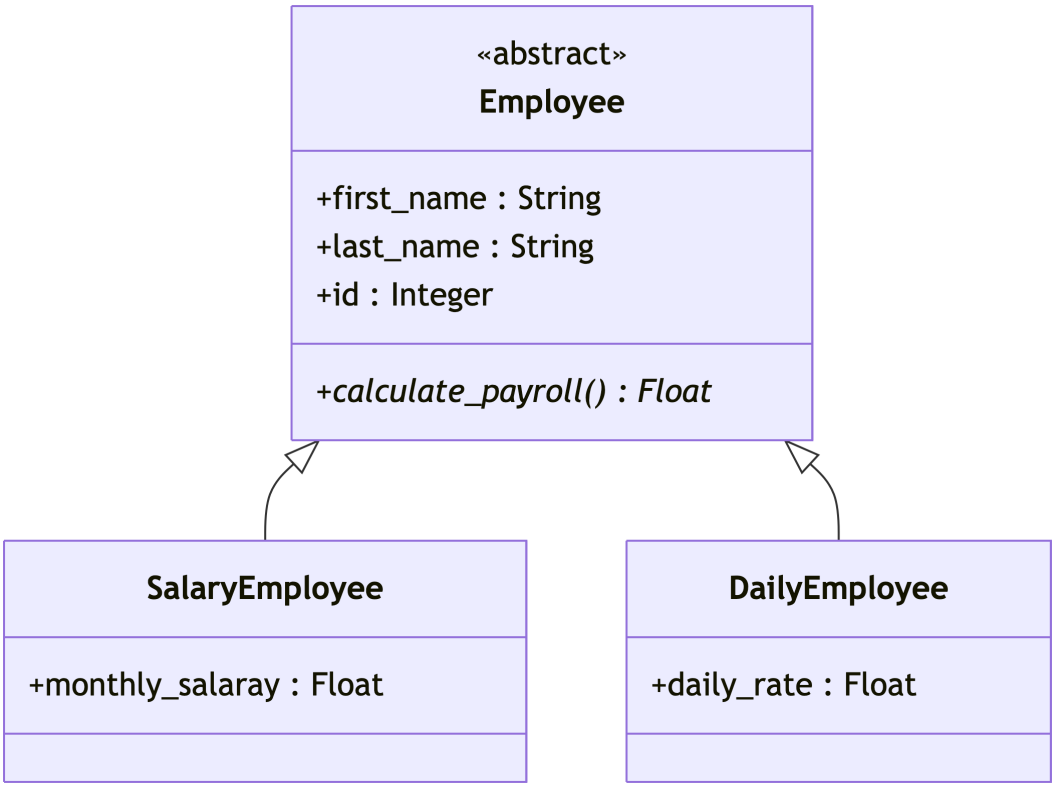
Retenez que les properties permettent d'exposer des attributs en masquant l'implémentation réelle. L'utilisateur de l'objet verra un attribut mais vous, vous serez libre de choisir son implémentation ou même la faire évoluer. Considérons en effet que cet objet évolue en intégrant une interaction avec une base de données. La property `students` accesseur ira chercher les informations en base. Il n'y a alors plus d'attribut `students` mais avec les properties, l'utilisateur de l'objet n'en a aucune idée.

Ces deux derniers exemples servent à vous illustrer que accesseurs et mutateurs ne sont pas associés à un attribut du même nom. La conception (ou l'évolution) d'une classe doit d'une part définir son *interface* (tout ce qui est publique) et d'autre part choisir au mieux l'implémentation.

# 9 Les classes et méthodes abstraites

La Programmation Orientée Objet possède le concept de classe abstraite. Une classe abstraite est une classe qui ne peut pas être instanciée. Elle possède une implémentation, un squelette de base, mais incomplet et l'objectif est que cette classe soit spécialisée.

Le schéma UML ci-contre dessous propose un exemple impliquant une classe abstraite dans notre contexte.




Dans notre système, chaque mois, je dois calculer la paye de chaque collaborateur. Pour cela, nous définissons pour la classe `Employee` une méthode, elle aussi abstraite, `calculate_payroll()`.

**Explications** : la classe `Employee` est trop générique pour implanter cette méthode car ce calcul dépends du statut du collaborateur, ici, un salarié payé au mois ou un prestataire payé sur la prestation (un prestataire n'est pas un employé en tant que tel mais on se permettra cette approximation pour l'illustration).

La mtéhode `calculate_payroll()` n'a donc pas d'implémentation ou une implémentation partielle et est déclarée comme abstraite ce qui entraine le statut de classe abstraite pour `Employee`.

Python ne gère pas nativement les classes abstraites. Python propose cependant la bibliothèque ABC pour Abstract Base Classes. La raison de son ajout est exposé dans la PEP 3119.

## 9.1 Exemple de classe abstraite

Vous pouvez déclarer une classe abstraite de la manière suivante.

```
1  class Employee(ABC):                                                    ①
2      def __init__(self, first_name: str, last_name: str, id:str):
3          self.first_name = first_name
4          self.last_name = last_name
5          self.id = id
6
7      @abstractmethod                                                     ②
8      def calculate_payroll(self) -> float:
9          pass
```

① Une classe abstraite hérite de `ABC`. **Elle doit contenir au moins une méthode abstraite**
② Une méthode est déclarée comme abstraite en étant décorée par `@absractmethod`. Cette méthode peut contenir du code.

Si vous essayez d'instancier cette classe, vous aurez une exception avec un message du type :

```
1  TypeError: Can't instantiate abstract class Employee with abstract method calculate_payrol
```

Vous devrez donc spécialiser cette classe.

## 9.2 Spécialiser une classe abstraite

Lorsque vous spécialisez une classe abstraite, si vous ne surchargez pas les méthodes abstraites, vous aurez toujours une classe abstraite.

Pour obtenir une classe concrète, vous devez implémenter toutes les méthodes abstraites comme dans l'exemple de code suivant.

```
class SalaryEmployee(Employee):

    def calculate_payroll(self) -> float:                    ①
        return self.monthly_salaray
```

① La méthode est spécialisée sans être décorée avec `@absractmethod` ce qui en fait une méthode concrète.

En complément, si la méthode générale contient du code, il est évidemment possible de l'appeler de la manière habituelle.

# 10 L'héritage multiple

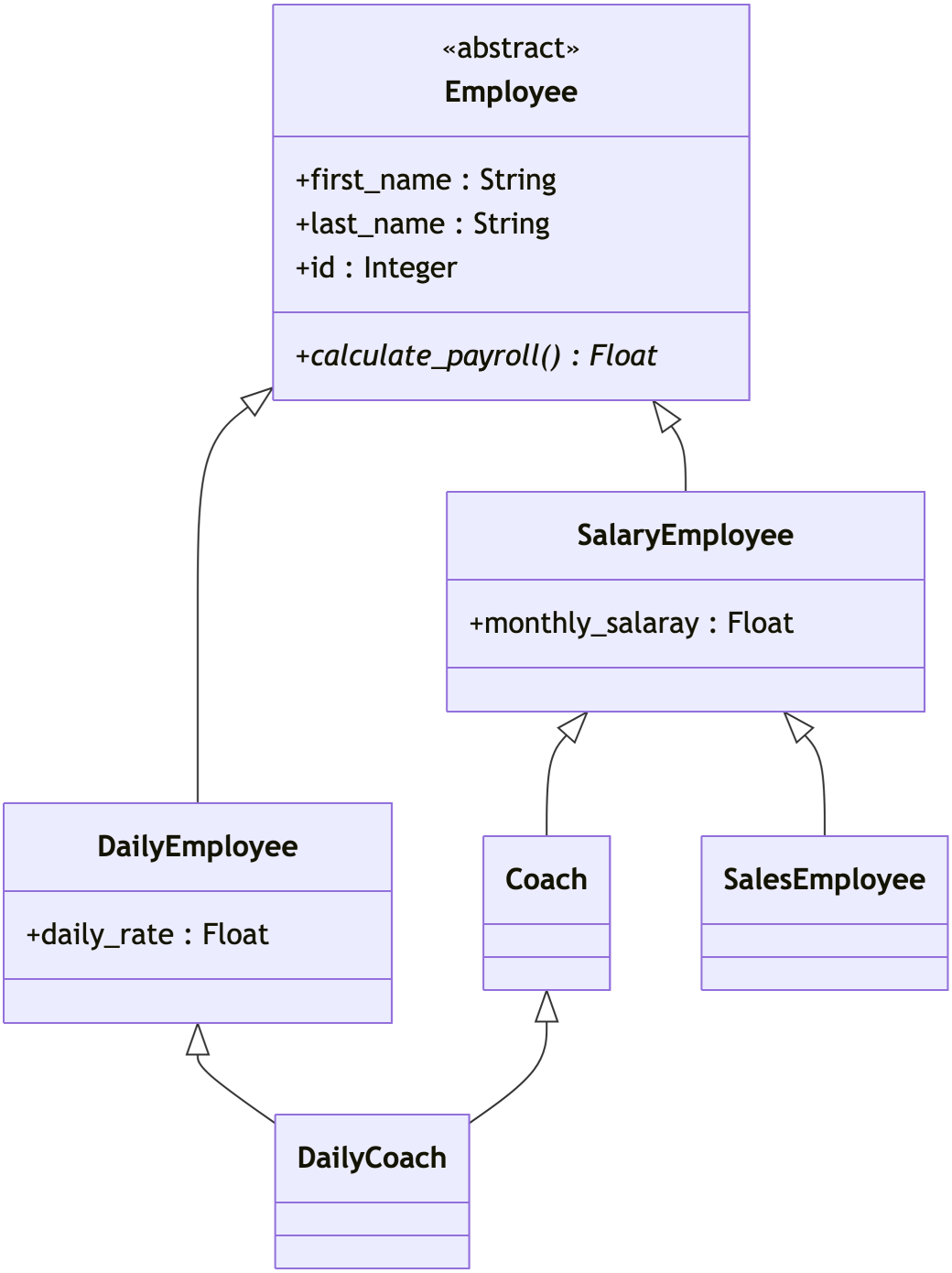
Python est un des rares langages modernes qui permette l'héritage multiple. Cela signifie qu'une classe peut spécialiser plusieurs classes.

Un grand principe de l'héritage multiple est… de l'éviter au maximum.

L'héritage multiple s'est révélé problématique ce qui a conduit la plupart des langages à s'orienter vers de l'héritage simple et à ajouter le concept d'interfaces. Dans cette partie, nous allons voir les principaux problèmes que nous pouvons rencontrer.

## 10.1 Modèle de travail

Le modèle es la partie employés d'un centre de formation reprenant celui de la partie précédente. Nous avions nos objets représentant des salariés et des prestataires. Nous avons spécialisé les salariés en commerciaux (`SalesEmployee`) et formateurs (salariés) (`Coach`).




L'implémentation des `SalaryEmployee` et `DailyEmployee` est la suivante :

```
1 class SalaryEmployee(Employee):
2     def __init__(self, first_name: str, last_name: str,
3                  id: int, monthly_salaray: float):
4
```

```
        super().__init__(first_name, last_name, id)                    ①
        self.monthly_salaray = monthly_salaray

    def calculate_payroll(self):                                       ②
        return self.monthly_salaray

class DailyEmployee(Employee):
    def __init__(self, first_name: str, last_name: str,
                 id: int, daily_rate: float):

        super().__init__(first_name, last_name, id)                    ①
        self.daily_rate = daily_rate
        self.days = 0                                                  ③

    def calculate_payroll(self):                                       ②
        return self.days * self.daily_rate
```

① La création de base de la classe est déléguée à `Employee`.
② Chaque classe a sa propre implémentation concrète de `calculate_payroll()`.
③ Permet de calculer le salaire, son évolution n'est pas montrée dans cet extrait de code.

Dans notre diagramme, nous avons une classe `Coach` qui spécialise `SalaryEmployee`. Elle n'est pas détaillée ici.

Nous aurons ensuite besoin de représenter un intervenant externe (`DailyCoach`). Ce `DailyCoach` est un `Coach` mais aussi un `DailyEmployee`. Dans la logique, il spécialise ces deux classes.

```
class DailyCoach(Coach, DailyEmployee):
    pass
```

Et là, nous allons commencer à avoir une série de problèmes résultant de la présence de mêmes fonctions dans les deux généralisations. Notez que le diagramme UML représente le modèle complet mettant en évidence le cas classique conduisant à ce type de problème : le Diamond Problem. En effet, nous ne voyons pas dans les classes au premier niveau que ces deux classes ont en fait un ancêtre commun.

## 10.2 L'ordre de résolution des méthodes

Lorsque qu'une méthode ou un attribut d'une classe est accédé, Python utilise son MRO (Method Resolution Order) pour le retrouver.

Une classe possède l'attribut spécial `__mro__` qui va retourner l'ordre dans lequel Python recherchera les attributs. Cet ordre est celui qui est respecté lorsque vous appelez la fonction `super()`. Dans le cas de `DailyCoach`, l'attribut retourne le résultat suivant.

```
In [1]: DailyCoach.__mro__
Out[1]:
(pyacademy.people.DailyCoach,
 pyacademy.people.Coach,
 pyacademy.people.SalaryEmployee,
 pyacademy.people.DailyEmployee,
 pyacademy.people.Employee,
 abc.ABC,
 object)
```

Lorsque nous créons une nouvelle instance, la méthode `__init__()` de `DailyCoach` est appelée ensuite celle de `Coach` puis celle de `SalaryEmployee` puis `DailyEmployee` et ainsi de suite.

## 10.3 Résolution du problème des paramètres

Si dans certains cas, appeler l'ensemble des méthodes sera ce que vous souhaitez, dans d'autres, vous ne le souhaiterez pas.

Dans nos exemples, le constructeur est un premier problème qui peut être résolu en appelant spécifiquement la méthode du parent souhaité.

```
class DailyCoach(Coach, DailyEmployee):
    def __init__(self, first_name: str, last_name: str,
                 id: int, daily_rate: float):

        DailyEmployee.__init__(self, first_name, last_name,
                               id, daily_rate)
```

Cependant, nous aurons à nouveau le problème lors de l'appel à .calculate_payroll(). Dans notre cas, il faudra à nouveau faire appel à l'implémentation spécifique.

## 10.4 Les mixins

Les mixins sont un moyen d'exploiter l'héritage multiple pour proposer un code réutilisable à la frontière entre les interfaces et les classes abstraites sans forcer à une hiérarchie rigide.

Une mixin est une classe qui va répondre à certaines conventions :

— Un nom destiné à éviter toute ambiguïté.
— Ce sont des classes *simples* qui réalisent un comportement. Elles n'ont pas de généralisation.
— Elles n'ont qu'une seule responsabilité

— Elles n'ont pas d'état donc rarement un constructeur ou des attributs d'instance ;
— Elles sont destinées à étendre le comportement d'autres classes et n'ont aucun sens par elles-même.

Un exemple de mixin peut être :

```python
class SerializableMixin:
    def serialize(self) -> dict:
        if hasattr(self, "__slots__"):
            return {
                name: getattr(self, name)
                for name in self.__slots__
            }
        else:
            return vars(self)
```

Django utilise intensivement les mixins pour étendre les comportements. Vous pouvez voir un exemple pour la validation du login : https://docs.djangoproject.com/en/5.2/topics/auth/default/#the-loginrequiredmixin-mixin ## Héritage multiple, conclusion

L'héritage multiple est source de nombreux problèmes, il est donc fortement déconseillé de l'utiliser. De manière générale, l'héritage est déconseillé en faveur de la composition. Il y a cependant une différence d'interprétation. L'héritage signifie **est un** alors que la composition signifie **contient/comporte un**. Nous pouvons le voir avec la modélisation d'une formation et plus spécifiquement une session de formation qui a lieu à une certaine date et à un certain lieu avec des stagiaires. Une formation (au sens sujet et contenu) n'est pas une liste, ce n'est pas non plus un lieu. Une session n'est pas non plus une formation (au sens sujet et contenu) ni un lieu. Par contre, une session comporte un sujet et un lieu.

Cependant, dans un langage déclaratif, il faut associer un type à nos classes pour informer le langage de ce que peut faire l'objet. Ces langages ont ajouté la notion **d'interfaces**. Une interface est un contrat déclarant ce que peut faire l'objet (c'est une *liste* de méthodes).

En Python, les interfaces sont inutiles. Les méthodes sont implémentés en fonction du besoin et les programmes n'ont pas besoin de connaitre ce que sont les objets (leur type) mais ce qu'ils peuvent faire (leurs méthodes).

Pour aller plus loin, n'hésitez pas à consulter ce post de Raymond Hettinger, *Python's super() considered super !* : https://rhettinger.wordpress.com/2011/05/26/super-considered-super/

Ainsi que celui de Guido Van Rossum sur Python History : http://python-history.blogspot.com/2010/06/method-resolution-order.html.

# 11 Les context managers

Un Context Manager définit un contexte dans lequel vous exécutez votre code. Le Context Manager fait parti de la famille des décorateurs et s'occupera donc d'exécuter des actions avant et/ou après votre code. Dans le contexte, vous vous concentrez sur votre métier et déléguez les questions technique au Context Manager.

Les context manager que vous avez dû voir sont la fonction `open()`, la fonction `pytest.raises()` (ou l'équivalente avec `unittest`) et l'objet connexion dans le cadre des bases de données. Peut-être aussi avec la fonction `os.scandir()`.

Nous allons voir dans cette partie comment créer nos propres context managers.

## 11.1 Présentation générale

Un Context Manager est une classe qui possède deux méthodes spéciales : `__enter__()` et `__exit__()`. Vous pourrez donc utiliser cette classe pour instancier un objet tout à fait normalement. Mais vous pourrez également utiliser cet objet avec le mot-clef `with`.

Le `with` appellera en premier lieu la méthode `__enter__()`. L'interpréteur exécutera ensuite votre code puis sortira du contexte en appelant la méthode `__exit__()`.

Voici ci-dessous l'implémentation la plus simple d'un context manager afin que vous voyez les différentes étapes d'appel. Notez que la méthode `__exit__()` prend des paramètres que nous verrons par la suite.

```
1 class BasicContextManager:
2     def __enter__(self):
3         print("Entering Context Manager")
4
5     def __exit__(self, exc_type, exc_val, exc_tb):
6         print("Exiting Context Manager")
```

Usage :

```
1 with BasicContextManager():
2     print('In context')
```

## 11.2 Un context manager reste un objet

Le contexte manager reste une classe et donc un objet. Il peut avoir une méthode `__init__(self)`

```
class BasicContextManager:
    def __init__(self):
        print("Initializing")
        self.my_id = "I'm the context"

    def __enter__(self):
        print("Entering Context Manager")

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exiting Context Manager")
```

Cette classe peut être instanciée et associée à une variable. La variable peut être utilisée pour sa fonction de contexte manager :

```
bc = BasicContextManager()
with bc:
    print(f'In context {bc.my_id}')
```

## 11.3 Idiome with as

Cet idiome permet d'instancier le contaxt manager au moment de son utilisation. Le mot clef `as` récupère le retour de la fonction `__enter__(self)`.

Ainsi, cette déclaration

```
class BasicContextManager:
    def __init__(self):
        print("Initializing")
        self.my_id = "I'm the context"

    def __enter__(self):
        print("Entering Context Manager")
        return self
```

Permet l'usage :

```
with BasicContextManager() as bc:
    print(f'In context {bc.my_id}')
```

De manière générale, la méthode `__enter__()` doit avoir un `return self` pour que l'idiome fonctionne. ## Gestion des exceptions

Une exception levée dans le contexte est interceptée par le context manager qui appelle alors la méthode `__exit__()` en passant des arguments liés à l'exception.

## 11.4 Paramètres de `__exit__()`

Si une exception se produit dans le contexte et l'interrompt, elle peut être gérée dans la méthode `__exit__()`. Les paramètres de cette méthodes vont alors entrer en jeu. La signature complète de la méthode est `__exit__(self, exc_type, exc_value, exc_tb)`. * **exc_type** : est le type (classe) de l'exception * **exc_value** : est l'instance de l'exception * **exc_tb** : est l'objet *traceback*

**Attention**, vous ne devez pas lever une exception dans ce bloc pour faire suivre l'exception.

## 11.5 Propagation de l'exception

Si vous avez géré l'exception et qu'elle ne doit pas être propagée, alors la méthode doit retourner une valeur **vrai** (**truthy**). Si la méthode retourne une valeur **falsy** (ce qui est fait par défaut puisque le retour est `None`), l'exception sera propagée.

# 12 Les itérateurs

Un itérateur est un objet qui permet de retourner le prochain élément d'un itérable. Les exemples les plus simples d'itérateurs sont les séquences mais aussi les générateurs.

Un **itérable** est un objet implantant la méthode spéciale `__iter__()` qui doit retourner un **itérateur**. Un **itérateur** est un objet implantant la méthode spéciale `__next__()`. Cette méthode doit retourner l'élément suivant ou lever une exception de type `StopIteration`.

## 12.1 Exemple de base

Nous allons utiliser notre formation pour illustrer la définition d'un itérateur. En effet, nous pouvons proposer d'accéder aux stagiaires par itération de type `for student in training`.

Nous allons donc ajouter ces deux méthodes à la classe `Training`. La méthode `__iter__()` retournera simplement `self`.

```
def __iter__(self):
    self._current_value = 0
    return self

def __next__(self):
    try:
        value = self.students[self._current_value]
        self._current_value += 1
        return value
    except IndexError:
        raise StopIteration("End of collection")
```

### 12.1.1 Limite du modèle de base

Ce choix d'implémentation n'est peut-être pas la plus pertinente mais elle permet d'illustrer que l'itérateur devra garder une trace de l'élément en cours.

Ceci révèle un problème dans cette implémentation : elle n'est pas adaptée aux tâches concurrente. Dans la pratique, la méthode `__iter__()` devrait retourner un objet dédié à l'itération et déconnecté de l'itérable.

### 12.1.2 D'autre outils d'itération

La bibliothèque itertools propose des fonctions créant des itérateurs pratiques : https://docs.python.org/3/library/itertools.html

## 12.2 Vers des fonctionnalité de collections

Nous avons fait de l'objet `TrainingSession` un itérable, mais nous pouvons aller plus loin et en faire un objet collection complet grâce à d'autres méthodes spéciales :

— `__len__()` est appelé par la fonction `len()` et doit retourner la taille de la collection
— `__contains__()` est appelé avec le mot clef `in` et doit retourner un booléen
— `__getitem__()` est appelé lors de l'écriture avec crochets

Bien entendu, ces méthodes ne doivent être ajoutée que si elles ont un sens avec votre classe. Un itérable ne sait pas forcément combien d'éléments il possède tout comme il peut ignorer la position réelle d'un élément.

Dans notre cas, vu que nous nous reposons sur une liste, nous pouvons ajouter l'ensemble de ces méthodes. Commençons par les plus simples.

```
def __len__(self):
    return len(self._students)

def __contains__(self, item):
    return item in self._students
```

La méthode `__getitem__(self, item)` est plus complexe. Le code suivant permet de voir quel est la valeur et le type du paramètre `item` :

```
def __getitem__(self, item):
    print(type(item))
    print(item)
```

Vous pouvez passer n'importe quel type à `__getitem__()` et vous récupérerez la donnée avec le type correspondant.

Mais si dans les crochets vous utilisez la syntaxe de slice, c'est bien un objet `slice` qui est passé en paramètre. Vous pourrez donc l'exploiter dans la méthode. Ceci peut être, en fonction de votre modèle, plus ou moins complexe. Dans notre cas, il ne s'agit que de *passe-plat*.

# 13 Les métaclasses

Les métaclasses sont un moyen performant, efficace et élégant de rajouter des fonctionnalités aux objets. Pour instancier un objet, deux phases sont nécessaires : la construction de l'objet, avec la méthode `__new__()` et l'initialisation de l'objet, avec la méthode `__init__()` qui sont deux points particulièrement distincts pour Python.

La seconde méthode est de loin la plus connue et utilisée, car dès que l'on décrit les paramètres passés au constructeur, il faut utiliser cette méthode, car cela fait partie de la phase d'initialisation.

La signature de cette méthode est donc la signature du constructeur, au premier argument près.

Lorsque l'on remonte l'arbre d'héritage, on trouve toujours tout à la fin l'objet object. Les instances ont un attribut `__class__` qui indique la classe liée à l'instance. Cette classe a elle-même un attribut `__class__` soit `instance.__class__.__class__` qui définit la metaclasse et par défaut, il s'agit de type.

Pour l'exemple ci-dessous, ABC a été retiré de l'arborescence d'héritage car il s'agit d'une métaclasse.

```
In [1]: from pyacademy.people import DailyCoach

In [2]: e = DailyCoach("John", "Doe", 52, 800)

In [3]: e.__class__
Out[3]: pyacademy.people.DailyCoach

In [4]: e.__class__.__class__
Out[4]: type
```

En Python, tout est objet. Les classes sont également des objets. Un objet ayant un type, les classes ont un type qui est `type`.

Et le type de type est… `type`.

type est une métaclasse.

## 13.1 Définir dynamiquement une classe

La fonction `type()`, lorsqu'elle reçoit un seul argument, retourne le type de l'objet ce qui correspond au même retour que `instance.__class__`.

Mais il est également possible de lui passer 3 arguments : `type(name, bases, dict)` ce qui créera une nouvelle instance de la métaclasse type. En d'autres termes, cela créera une nouvelle classe.

- `name` définit le nom de la classe et deviendra l'attribut `__name__`
- `bases` est un tuple des classes dont hérite cette classe.
- `dict` est un dictionnaire qui contient la définition des attributs et méthodes, c'est le *dictionnaire du namespace*.

Nous pouvons donc définir un nouveau type le plus simple de la manière suivante.

```
In [1]: Foo = type('Foo', (), {})

In [2]: f = Foo()

In [3]: f
Out[3]: <__main__.Foo at 0x102809060>

In [4]: type(f)
Out[4]: __main__.Foo
```

Un autre exemple avec un attribut et héritant de la définition précédente :

```
In [7]: Bar = type("Bar", (Foo,), {"attr":100, "val_bar":lambda x:x.attr})

In [8]: b = Bar()

In [9]: b.attr
Out[9]: 100

In [10]: b.__class__
Out[10]: __main__.Bar

In [11]: Bar.__mro__
Out[11]: (__main__.Bar, __main__.Foo, object)

In [12]: b.val_bar()
Out[12]: 100
```

## 13.2 Créer nos propres metaclasses

Pour créer nos propres metaclasses, nous devons avoir une classe héritant de type. La structure et le code type sera le suivant.

```
class MyMetaClasse(type):
    def __call__(self, *argv, **kwargs):
        print("Dans call METACLASSE")
        return super().__call__(*argv, **kwargs)

    def __new__(cls, name, base, dct):
        print("Dans new METACLASSE")
        return super().__new__(cls, name, base, dct)

    def __init__(self, *argv, **kwargs):
        print("Dans init METACLASSE")
        super().__init__(*argv, **kwargs)
```

Cette metaclasse peut maintenant être utilisée de la manière suivante lors de la création de notre classe.

```
class MyClass(metaclass=MyMetaClasse):
    pass
```

En conclusion, vous n'auriez que très rarement besoin de créer des metaclasses. Leur usage reste ésotérique et il y aura certainement une solution plus simple et  plus maintenable. La citation ci-contre de Tim Peters, le guru Python auteur du Zen de Python, résume leur intérêt.

> Metaclasses are deeper magic than 99% of users should ever worry about. If you wonder whether you need them, you don't (the people who actually need them know with certainty that they need them, and don't need an explanation about why)
>
> *Tim Peters*

Cependant, même si vous n'utiliserez certainement jamais les metaclasses, leur connaissance reste importante pour la compréhension du modèle objet.

# Cinquième partie

# Initiation au calcul distribué

Lorsqu'un programme augmente en complexité, il se retrouve avec le besoin d'exécuter des traitements en même temps. Nous aurons alors à gérer le problème de concurrence. Mais un processeur ne peut exécuter qu'une seule tâche à la fois. Il va donc avancer les différentes tâches de manière successive. Python propose deux outils pour gérer ce cas : threading qui permet du multi-tâches préemptif et asyncio qui permet du multi-tâches coopératif.

Le parallélisme consiste à exécuter plusieurs traitements en même temps sur plusieurs processeurs ou sur plusieurs cœurs. La programmation parallèle est indispensable dans certains domaines comme le calcul scientifique.

## Terminologie

Un **processus** est l'exécution d'un ensemble d'instructions par l'utilisation de ressources physiques, les deux principales étant la mémoire vive et le processeur. Un programme peut correspondre à un seul ou plusieurs processus. Si un programme est multi-processus, chacun dispose de son propre environnement d'exécution et est indépendant. Cette technologie est idéale lorsque l'on dispose de plusieurs processeurs.

D'un point de vue utilisateur, une **tâche** (ou **thread**) présente les mêmes caractéristiques qu'un processus. Mais toutes les tâches utilisent le contexte d'exécution du processus auquel elles appartiennent. Elle possèdent que leur propre pile d'appels. Le parallélisme entre tâches n'est pas réel.

Du fait de l'implémentation de CPython, Python ne peut exécuter qu'un seul thread à la fois. Ceci est verrouillé par un outil nommé GIL (Global Interpreter Lock) et vous pouvez trouver plus d'informations sur les limites dans la présentation suivante : http://www.dabeaz.com/python/UnderstandingGIL.pdf.

Python propose une API de haut niveau nommée `threading` pour gérer les threads. Elle n'apportera un gain de performances que pour les tâches qui vont attendre des évènements externes. Cependant, utiliser les threads peut aussi être une manière d'améliorer le design de l'application.

# 14 Introduction à threading

Créer un thread se fait par la création d'un objet de type `threading.Thread()`.

La signature de la classe est la suivante (tous les paramètres ne sont pas représentés) :

```
threading.Thread(target=None, args=(), kwargs={}, daemon=None)
```

Les arguments doivent être nommés.

- `target` : objet *callable* qui sera appelé lors de l'exécution du thread
- `args` : séquence d'arguments positionnels pour l'appel du callable
- `kwargs` : séquence d'arguments nommés pour l'appel du callable
- `daemon` : Définit explicitement si le thread est détaché du fil courant.

Pour illustrer, nous allons nous utiliser des actions *longues* et pour cela simplement mettre l'exécution du code en attente. Pour l'observer, nous utiliserons un logger.

Nous allons utiliser (et donc importer) les bibliothèques `logging`, `threading` et `time`.

```
def thread_function(name):
    logging.info(f"Thread {name} started")
    time.sleep(2)
    logging.info(f"Thread {name} finished")
```

Nous pouvons créer un thread pour avec cette fonction. Pour exécuter ce thread, il suffit de faire un appel à la méthode `Thread.start()`.

```
logging.info("Main    : before creating thread")
x = threading.Thread(target=thread_function, args=(1,))
logging.info("Main    : before running thread")
x.start()
logging.info("Main    : wait for the thread to finish")
logging.info("Main    : all done")
```

Si vous exécutez ce programme, vous verrez qu'il se termine avant le thread.

## 14.1 Les threads en tant que daemon

En informatique, un daemon est un processus qui s'exécute en tâche de fond.

Pour threading, un daemon est un thread qui sera arrêté à la fin du programme, qu'il ai terminé son exécution ou non.

Ainsi, si vous complétez la déclaration du thread de la manière suivante :

```
x = threading.Thread(target=thread_function, args=(1,), daemon=True)
```

Vous ne verrez pas l'affichage du dernier log.

## 14.2 Synchroniser les thread

La méthode .join() de l'objet thread fait attendre au fil principale la fin de l'exécution du thread.

Insérée dans le code précédent, le programme s'arrête le temps que le thread se termine.

```
x.start()
logging.info("Main    : wait for the thread to finish")
x.join()
logging.info("Main    : all done")
```

## 14.3 Travailler avec plusieurs threads

Si vous exécutez plusieurs threads à partir d'un même thread principal, vous verrez que, potentiellement, les exécutions peuvent se croiser. Un thread peut, pour la même exécution, se terminer avant un autre thread démarré avant.

C'est le système qui détermine quel thread faire avancer. Vous n'avez pas de moyen d'anticiper quel thread le système fera avancer.

Si les threads sont indépendants, il n'y a pas vraiment de conséquence. Par contre, nous pouvons avoir à faire face à des *race conditions*.

## 14.4 Les *race conditions*

Une *race condition* se traduit par *situation de compétition* ou *situation de concurrence*. Il s'agit de concurrence d'accès à une ressource.

L'illustration suivante montre le problème de gestion d'un compteur. Le compteur est géré par la classe suivante.

```
class SomeService:
    def __init__(self):
        self.value = 0

    def update(self, name):
        logging.info(f"Thread {name}: starting update")
        local_copy = self.value
        local_copy += 1
        time.sleep(0.1)
        self.value = local_copy
        logging.info(f"Thread {name}: finishing update")
```

Dans notre programme principal, nous aurons un seul service. Puis, nous allons déclencher deux threads qui mettent à jour le compteur.

```
service = SomeService()

t1 = threading.Thread(target=service.update, args=(1,))
t2 = threading.Thread(target=service.update, args=(2,))
logging.info(f"Testing update. Starting value is {service.value}.")
t1.start()
t2.start()
t1.join()
t2.join()
logging.info(f"Testing update. Ending value is {service.value}.")
```

Les logs correspondant sont présentés ci-dessous, la valeur finale de 1 aurai dû être à 2.

```
Testing update. Starting value is 0.
Thread 1: starting update
Thread 2: starting update
Thread 1: finishing update
Thread 2: finishing update
Testing update. Ending value is 1.
```

Nous allons avoir besoin de synchroniser nos threads et une manière élémentaire pour le faire est de créer un *verrou*.

### 14.4.1 Synchronisation avec lock

Un verrou agira comme un totem. Il ne peut être possédé que par un seul thread. Pour qu'un autre thread puisse l'avoir, il faut que le précédent le restitue.

Pour cela, les méthodes de base sont .acquire() et .release(). Si un thread fait appel à .acquire() mais que le lock n'est pas disponible, il attendra jusque'à sa libération.

Les verrous peuvent être utilisés comme un context manager.

Notre classe devient le code ci-dessous.

```
class SomeService:
    def __init__(self):
        self.value = 0
        self._lock = threading.Lock()

    def update(self, name):
       with self._lock:
            local_copy = self.value
            local_copy += 1
            time.sleep(0.1)
            self.value = local_copy
```

Si vous ajoutez du log autour du context manager, vous verrez que le second thread commencera l'appel à la méthode avant la fin de l'exécution du premier thread. Mais il sera bloqué avant le contexte manager tant que le premier thread n'a pas relâché ce lock.

Ceci permet de résoudre le problème d'accès concurrent aux ressources partagées entre les threads.

### 14.4.2 Deadlock

Le lock amène un autre problème : deux threads peuvent se bloquer mutuellement. C'est le principe des deux coffres dans lesquels sont enfermées les clefs pour ouvrir l'autre. C'est une situation de **deadlock**.

Un deadlock survient communément dans une des deux situations suivantes :

— Un bug d'implémentation où le lock n'est pas libéré proprement
— Un design où la fonction a besoin d'être appelée par une fonction qui peut avoir ou non un lock. Cette situation arrive habituellement dans les situations d'appels récursifs.

Pour éviter la première situation, préférez la syntaxe avec le contexte manager.

Pour la seconde, Python propose un second objet, RLock. Il permet à un même thread d'acquérir un même lock plusieurs fois avant de le libérer. Cette classe est conçue pour les cas des appels récursifs.

En dernier recours, vous avez l'option du paramètre `timeout` de la méthode `.acquire()`. L'argument doit être un réel correspondant à un nombre de secondes.

## 14.5 Limiter le nombre de threads avec les sémaphores

Le sémaphore est une des plus anciennes primitives de synchronisation conçue par l'informaticien Edsger W. Dijkstra. Un sémaphore est initialisé avec un compteur et expose deux méthodes : `.acquire()` et `.release()`.

> Edsger W. Dijkstra avait défini pour les sémaphores les opérations `Init`, `P` et `V`. `P` et `V` viennent du néerlandais *Proberen* et *Verhogen* qui signifient respectivement *Sonder* et *incrémenter*.

Un appel à la méthode `.acquire()` décréments le compteur. Un appel à la méthode `.release()` incrémente le compteur.

Un appel à `.acquire()` lorsque le compteur est à 0 bloquera le thread qui a fait cet appel jusque'à ce qu'un autre fasse appel à `.release()`.

La valeur passée à l'initialisation n'est pas un maximum. Il est possible de faire appel à `.release()` alors que le sémaphore est à cette valeur et donc l'augmenter. La classe `threading.BoundedSemaphore` est identique à `threading.Semaphore` mais empêche que le compteur excède sa valeur initiale.

Les sémaphores sont utilisés pour limiter le nombre d'utilisation simultanée d'une ressource comme par exemple l'accès à une base de données.

C'est également la solution au problème du diner des philosophes

## 14.6 Spécialiser les Threads

La classe `threading.Thread` peut être spécialisée avec l'héritage. Mais pour le faire correctement, il est indispensable de connaitre le fonctionnement d'un objet `Thread`.

Dans les exemples précédents, nous avons fait appel, pour démarrer le thread, à la méthode `Thread.start()`. Cette méthode crée un nouveau thread et appelle la méthode `Thread.run()`.

Lorsque nous consultons le code source de cette méthode, nous avons (documentation omise) :

```
def run(self):
    try:
        if self._target is not None:
            self._target(*self._args, **self._kwargs)
    finally:
        del self._target, self._args, self._kwargs
```

Cette méthode appelle donc le *callable* passé en argument à la création de l'objet en lui passant en argument les éléments de args et kwargs.

Vous comprenez donc que vous pouvez créer plus spécifiquement un thread qui ne recevra pas de *callable* mais aura directement le code dans la méthode `.run()`.

Créer son propre thread consiste donc à spécialiser cette classe et à surcharger le constructeur et la méthode `.run()`. Vous ne devriez pas surcharger une autre méthode. Cependant, vous pouvez, et c'est pourquoi vous créerez vos propres threads, ajouter d'autres méthodes.

# 15 Introduction au multiprocessing

Le module `multiprocessing` tire tous les avantages des CPUs. À un haut niveau, il crée de nouvelles instances de l'interpréteur pour fonctionner sur un CPU dédié. Lancer un nouvel interpréter est plus couteux que démarrer un nouveau thread.

Le module `multiprocessing` a la même interface que le module `threading`. De base, vous ne pourriez n'avoir que l'import à changer en remplaçant `Thread` par `Process`. Une astuce consiste à utiliser un alias pour l'import afin de switcher l'import à loisir.

Notez également qu'avant d'utiliser les processus fils, sous Windows, devez ajouter cette ligne.

```
1 from multiprocessing import freeze_support
2 if __name__ == "__main__":
3     freeze_support()
```

L'explication vient du fait que sous Windows, le fork d'un programme n'est pas possible de la même façon que sous les systèmes Posix. Il est d'usage d'ajouter systématiquement cette ligne qui sera sans effet sous Linux/MacOS.

Cependant, utiliser plusieurs process revient à *forker* votre programme, les données ne sont alors plus partagées. Ainsi, à chaque nouveau process, vous aurez un *nouveau* programme avec une copie des données.

Pour manipuler les même données il faut les allouer à une mémoire partagée.

## 15.1 Utilisation de la mémoire partagée

Utiliser la mémoire partagée comporte quelques limites.

Les types de Python (dictionnaires, listes, …) ne sont pas utilisables dans la mémoire partagée. Seuls les types du module `ctypes` le sont. Il s'agit des types natifs du langage C. Le module multiprocessing permet d'allouer ces données via 4 classes :

— `RawValue`, donnée brute, sans verrou associé `data = RawValue(ctypes.c_long, 0)`
— `RawArray`, tableau brut, sans verrou associé
— `Value`, valeur ayant un verrou qui lui est associé `data = Value(c_types.c_long, 0, lock=True)`
— `Array`, tableau ayant un verrou associé.

Une valeur issue d'une variable de type `Value`/`RawValue` ne peut pas être manipulée avec une variable ordinaire Python. Il faut utiliser la propriété `.value` comme dans l'exemple suivant :

```
In [17]: n = RawValue('d', 10)                                          ①

In [18]: n.value
Out[18]: 10.0

In [19]: n.value += 10

In [20]: n.value
Out[20]: 20.0
```

① l'argument `'d'` représente un `c_double`. Vous pouvez également utiliser `'i'` pour les `c_int` et `'h'` pour les `c_short`.

Le verrou des objets de type Value/Array s'utilise via la méthode `get_lock()` et sont *context safe*.

```
In [10]: counter = Value("i", 0)

In [11]: with counter.get_lock():
    ...:     counter.value += 1
```

# 16 Le module `concurrent.futures`

`concurrent.futures` est le module Python qui permet d'exécuter du code en parallèle de façon simple, avec une interface unifiée pour les threads et les processus. Le module fournit 2 *Executors* :

— `ThreadPoolExecutor` qui gère les threads donc pour les tâches I/O-bound, idéal pour les appels réseau, lectures de fichiers, requêtes HTTP.
— `ProcessPoolExecutor` qui gère les process donc pour les tâches CPU-bound, chaque worker tourne dans un processus Python distinct.

## 16.1 Exemple

L'exemple va récupérer les statuts de sites web. La mise en place est la suivante :

```
from concurrent.futures import ThreadPoolExecutor
import urllib.request

urls = [
    "https://example.com",
    "https://python.org",
    "https://pypi.org",
]

def fetch(url):
    with urllib.request.urlopen(url, timeout=5) as r:
        print(f"{url} : {r.status}")
```

Le `ThreadPoolExecutor` s'utilise ainsi :

```
with ThreadPoolExecutor(max_workers=2) as executor:                    ①
    for url in urls:
        executor.submit(fetch, url,)                                   ②

```

① Création de l'exécutor et usage en context manager afin de garantir la fin de toutes les tâches.
② L'appel à `.submit()` crée un thread et l'ajoute à la file de l'executor.

Dans cet exemple, l'Executor a été paramétré avec `max_workers=2` ce qui fait qu'il lance deux threads en parallèle au maximum.

`executor.submit()`

```
def fetch(url):
    with urllib.request.urlopen(url, timeout=5) as r:
        return url, r.status

with ThreadPoolExecutor(max_workers=2) as executor:
    for url, status in executor.map(fetch, urls):
        print(f"{url} : {status}")
```

# 17 Le problème des producteurs et consommateurs

Le problème des producteurs et consommateurs, ou producer-consumer problem, est un cas de synchronisation de ressources. Le producteur génère des données à traiter par le consommateur. Il y aura donc une file d'attente entre la génération et le traitement des données qui ont donc lieu dans deux *tâches* différentes.

Python fournit le module queue pour la communication entre threads et le module multiprocessing comporte des implémentations similaires pour le multiprocessing.

Dans ce document, seule l'usage avec les threads est abordé mais le principe reste le même avec le multiprocessing.

## 17.1 Illustration

Le cas d'usage est le suivant :

```
1  import threading
2  from queue import SimpleQueue                                    ①
3
4  def producer(q):
5      for i in range(5):
6          q.put(i)                                                 ②
7          print(f"Produit : {i}")
8      q.put(None)                                                  ④
9
10 def consumer(q):
11     while True:
12         item = q.get()                                           ③
13         if item is None:
14             break
15         print(f"Consommé : {item}")
16
17 q = SimpleQueue()
18 t1 = threading.Thread(target=producer, args=(q,))
19 t2 = threading.Thread(target=consumer, args=(q,))
20
```

```
21 t1.start(); t2.start()
22 t1.join(); t2.join()
```

① Nous utilisons ici l'objet `SimpleQueue`
② `q.put()` ajoute un élément à la file
③ `q.get()` attends qu'un élément soit disponible
④ Définition d'une *sentinelle* pour signaler la fin

## 17.2 Différents types de files

Le module `queue` propose plusieurs implémentations.

La classe `queue.Queue` est la plus complète et permet de gérer la synchronisation avec `.join()` et une taille maximale. Ses méthodes usuelles sont :

— `.empty()` retourne vrai si la liste est vide
— `.full()` retourne vrai si la liste est pleine
— `.get(block=True, timeout=None)` retourne le prochain élément
— `.put(item, block=True, timeout=None)` ajoute un élément.
— `.join()` met en attente de tous les éléments de la liste.

Pour des cas plus simples, le module propose `queue.SimpleQueue` est une version allégée correspondant à la plupart des cas d'usage FIFO. Elle possède les méthodes précédentes sauf `.join()`.

## 17.3 La notion de sentinelle

La méthode `.get()` est bloquante. Elle attends la prochaine valeur à traiter et toute valeur est traitée de la même manière. Il est alors convenu d'une valeur dite *sentinelle* qui sera transmise pour indiquer qu'il n'y a plus d'éléments.

## 17.4 Queues, files et autres implémentations

Notez qu'une *queue* en informatique représente un type de pile, les FIFO (First In First Out). Une pile de type LIFO (Last in First Out) est nommée une *stack*. L'objet Queue a bien le comportement qui correspond à son nom.

La bibliothèque propose d'autres types

— `LifoQueue` pour les stacks
— `PriorityQueue` pour sortir les éléments par ordre de priorité.

La priorité des éléments dans une `PriorityQueue` est définie par leur ordre naturel. Il n'est donc pas rare de créer une `DataClass` pour gérer les éléments de ce type de queue.

Ces deux classes possèdent les mêmes méthodes que `Queue`. Leur différence est dans l'ordre d'accès aux éléments.

# 18 Programmation parallèle avec Asyncio

Le principe d'`asyncio` est de permettre à Python d'exécuter une autre tâche pendant que l'opération actuelle attends. Il s'agit de multitâche coopératif. Ceci signifie que c'est la tâche qui décide quand rendre la main et non le système.

Le concept général est qu'un objet unique appelé la boucle d'évènements (event loop) contrôle quand et comment chaque tâche est exécutée. Cette boucle connait chaque tâche et leur état. On peut simplifier ces états en *prêt à être exécuté* ou *en attente*. La boucle connaissant les états replace une tâche qui a terminée dans la liste correspondant à son état et démarre la suivante en attente.

Le point important est que c'est la tâche qui rend la main, ce qui signifie qu'elle ne sera pas interrompue en milieu d'exécution.

Le cœur de `asyncio` est la coroutine. Ceci s'accompagne de deux mots-clef : `async` et `await`.

## 18.1 Déclaration d'une coroutine

On déclare une coroutine avec les mots clef `async def`.

```
1 async def my_coroutin():
2     print("Je suis une coroutine")
```

Exécuter cette fonction retourne un objet de type `<coroutine object my_coroutin at ...>`. Cet objet devra être exécuté dans la boucle d'évènements d'`asyncio`.

## 18.2 Rendre la main

La coroutine précédente n'est pas différente d'une fonction, `asyncio` l'exécutera dans son ensemble car elle ne *rends* pas la main. Pour cela, il faut déclarer que la coroutine attends le résultat d'une opération avec le mot clef `await` :

```python
import asyncio

async def count(name:str):
    print(f'{name} One')
    await asyncio.sleep(1)
    print(f'{name} Two')
    return f'{name} done`
```

Notez que l'on utilise `asyncio.sleep()` qui permet de suspendre la coroutine et non `time.sleep()`qui bloque le thread.

## 18.3 Créer des tâches

Une seule coroutine ne permet pas des opérations en parallèle. Le contenu de la coroutine reste séquentiel. L'intérêt est lorsque la boucle d'évènements exécute plusieurs coroutines sous forme de tâches.

### 18.3.1 Utiliser `asyncio.create_task()`

```python
async def main():
    c1 = asyncio.create_task(count(1, 3))
    c2 = asyncio.create_task(count(2, 1))
    c3 = asyncio.create_task(count(3, 2))

    print("Les trois tâches sont lancées")

    r1 = await c1
    r2 = await c2
    r3 = await c3

    print(r1, r2, r3)


asyncio.run(main())
```

Les trois tâches débuteront à peu près en même temps, cependant, l'ordre du second affichage ne sera pas forcément celui de lancement des tâches. Le temps d'exécution total sera de l'ordre de 3 secondes.

Notez que `asyncio.create_task()` ne démarre pas de tâche, il la planifie pour plus tard et c'est les lignes 8 à 10 qui permettent leur exécution dans la boucle d'évènements.

### 18.3.2 Utiliser `asyncio.gather()`

L'usage précédent permet de référencer les tâches et les gérer individuellement. `asyncio.gather()` permet quand à lui d'attendre plusieurs opérations asynchrones comme un groupe.

```
async def main():
    await asyncio.gather(count(),
                         count(),
                         count(),
                        )
```

`asyncio.gather()` retourne une données qui une liste des résultats.

# Sixième partie

# Initiation à la Data Science

Les supports liés à l'initiation aux Data Science et Machine Learning sont sous form de Notebooks Jupyter dans un projet dédié qui vous sera communiqué lors de la formation.

# Septième partie

# Environnement de travail et packaging

# 19 Structure de projet Python

Deux approches de référence pour organiser un projet Python selon son usage : la proposition de **Kenneth Reitz** (projets applicatifs) et la structure **src-based** (packaging et distribution).

---

## 19.1 Structure proposée par Kenneth Reitz

Popularisée par le guide The Hitchhiker's Guide to Python, cette structure convient aux projets applicatifs, scripts, outils CLI et services web. Elle privilégie la lisibilité et la convention sur la configuration.

```
mon_projet/
+-- mon_projet/                    (1)
|   +-- __init__.py
|   +-- __main__.py                (2)
|   +-- core.py
|   \-- ...
|
+-- tests/                         (3)
|   +-- __init__.py
|   +-- test_core.py
|   \-- ...
|
+-- docs/                          (4)
|   +-- conf.py
|   \-- index.rst
|
+-- .env.example                   (5)
+-- .gitignore
+-- LICENSE                        (6)
+-- Makefile                       (7)
+-- README.md                      (8)
+-- requirements.txt               (9)
+-- requirements-dev.txt           (10)
\-- pyproject.toml                 (11)
```

① Package *principal*, vous ne devez théoriquement avoir qu'un seul package pour votre projet principalement pour imposer un **namespace**.
② Pour rappel, un fichier `__main__.py` est le point d'entrée en *exécutant le module* (ici par un `python -m mon_projet`)
③ Le package de tests isolés du code métier.
④ Documentation (Sphinx, MkDocs…).
⑤ Optionnel, variables d'environnement (modèle).
⑥ Il est recommandé de toujours préciser la licence d'utilisation.
⑦ Spécifique au monde Posix et à l'outil `make` pour automatiser les tâches courantes (test, lint, run…).
⑧ Le `README.md` doit présenter le projet et indiquer comment configurer l'environnement de dev.
⑨ Dépendances runtime
⑩ Dépendances de développement
⑪ Optionnel, norme récente pour centraliser les métadonnées (PEP 621)

### 19.1.1 Principes clés

— Le package applicatif est **à la racine** du dépôt, directement importable après `pip install -e .`
— `tests/` est un répertoire de premier niveau, **séparé** du package
— Pas de sous-dossier `src/` : le code est découvrable sans configuration supplémentaire
— Adapté aux projets **Django**, **Flask**, **FastAPI**, scripts de data science, outils internes

### 19.1.2 Exemple d'import

```
# Après pip install -e .
from mon_projet.core import MaClasse
```

---

## 19.2 Structure src-based

Recommandée par PyPA et adoptée comme bonne pratique pour les bibliothèques distribuées sur PyPI. Elle isole le code source dans un répertoire `src/` afin d'éviter les importations accidentelles depuis la racine du projet lors des tests.

```
mon_projet/
+-- src/
|   \-- mon_projet/                                                    ①
|       +-- __init__.py
```

```
|       +-- __main__.py
|       +-- core.py
|       \-- ...
|
+-- tests/                                                    (2)
|   +-- conftest.py
|   +-- test_core.py
|   \-- ...
|
+-- docs/
|   \-- ...
|
+-- .gitignore
+-- LICENSE
+-- README.md
+-- pyproject.toml                                            (3)
+-- requirements.txt                                          (4)
\-- requirements-dev.txt
```

① Package installable
② Tests à la racine, séparés du package *source*
③ Configuration unique (build, deps, outils)
④ La présence des requirements est destiné aux devs (de manière historique)

### 19.2.1 `pyproject.toml` minimal

À titre d'exemple, voici le contenu d'un fichier `pyproject.toml` minimal qui référence `setuptools`.

```toml
[build-system]
requires = ["setuptools >= 77.0.3"]
build-backend = "setuptools.build_meta"

[project]
name = "mon-projet"
version = "0.1.0"
description = "Description courte du projet"
readme = "README.md"
requires-python = ">=3.11"
license = { file = "LICENSE" }
dependencies = [
    "httpx>=0.27",
]

```

```toml
[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "ruff>=0.4",
]
```

### 19.2.2 Principes clés

— Le code dans `src/` **n'est pas importable** sans installation préalable (`pip install -e .`), ce qui garantit que les tests exercent bien le package installé et non les fichiers locaux bruts
— Un seul fichier de configuration : `pyproject.toml` (remplace `setup.py`, `setup.cfg`, `MANIFEST.in`)
— `tests/` reste **en dehors** de `src/` pour ne pas être inclus dans la distribution
— Adapté à la **publication sur PyPI**, aux bibliothèques réutilisables, aux projets open source

### 19.2.3 Installation en mode éditable

```
pip install -e ".[dev]"
```

---

## 19.3 Références

— The Hitchhiker's Guide to Python – Structuring Your Project
— PyPA – Packaging Python Projects
— PyPA – src layout vs flat layout
— Hatch – Build backend moderne
— PEP 517 – Build system interface

# 20 Les outils de développement

## 20.1 Isoler les environnements : venv

Afin de travailler dans un environnement stabilisé sans ambiguïté sur l'interpréteur utilisé et avec des dépendances maitrisées, Python propose d'isoler la configuration de l'environnement avec les environnements virtuels.

## 20.2 Création d'un environnement virtuel

```
1 python -m venv .venv
```

- `python` : utiliser la version explicite de la version qui sera la version du venv (par exemple `python3.14`).
- `-m venv` : exécution de `venv`.
- `.venv` : chemin du répertoire de création du venv, ici `.venv` dans le répertoire courant.

Vous pouvez créer le répertoire contenant le venv n'importe où. Le créer dans le répertoire du projet permet d'avoir un venv dédié au projet et le nom `.venv` est convenu pour utiliser la propriété du fichier caché sous les environnements POSIX.

## 20.3 Usage et activation de l'environnement virtuel

Précisions : dans l'arborescence de l'environnement virtuel, les *exécutables* sont dans un sous-répertoire dépendant du système : `bin` sous Linux et MacOs, `Scripts` sous Windows.

Les environnements virtuels sont destinés à être utilisés dans une invite de commande et configureront le process. Ils sont intégrés dans les principaux IDEs mais leur usage n'est pas couvert ici.

### 20.3.1 Activation d'un environnement virtuel

Voir la documentation

Sous Linux ou MacOs

```
source .venv/bin/activate
```

Sous windows, selon que l'on soit sous le PowerShell ou le cmd :

```
.venv\Scripts\Activate.ps1
.venv\Scripts\activate.bat
```

Suite à cette activation, vous utiliser `python` et `pip` de l'environnement virtuel. ### Utilisation directe des exécutables

Pour un usage simple, vous pouvez utiliser directement les exécutables de l'environnement virtuel (pour Linux ou MacOs) :

```
.venv/bin/pip list
.venv/bin/python
```

## 20.4 Gérer les dépendances : `pip`

`pip` est l'outil standard de gestion des dépendances. Si vous travaillez avec un environnement virtuel (conseillé), vous devez l'activer auparavant.

Les instructions principales sont :

### 20.4.1 Lister les packages installés

```
pip list
```

### 20.4.2 Créer une référence des dépendances

La commande `pip freeze` affiche les dépendances de l'environnement courant dans le terminal. Elle est utilisée pour créer le fichier informant des besoins de ce projet :

```
pip freeze > requirements.txt
```

### 20.4.3 Installer des dépendances

```
pip install my_package                                    ①
pip install -r requirements.txt                           ②
```

① Installe le package en argument
② Utiliser le fichier `requirements.txt` pour installer l'ensemble des dépendances nécessaires au projet.

`pip` permet de passer par un proxy, vous devez alors connaitre l'adresse du proxy et utiliser une syntaxe du type :

```
python -m pip install --proxy user:pass@proxy.server:port my_package
```

## 20.5 uv, Gestionnaire de projet Python *moderne*

**uv** est un outil écrit en Rust, développé par Astral (auteurs de Ruff).
Il remplace en un seul binaire : `pip`, `pip-tools`, `virtualenv`, `pyenv`, `pipx` et partiellement `poetry`.

L'installation ne sera pas abordée ici car spécifique des systèmes, voir la documentation

## 20.6 Créer un projet

```
uv init mon_projet
cd mon_projet
```

Structure générée :

```
mon_projet/
   .python-version # <1>
   pyproject.toml # <2>
   uv.lock # <3>
   README.md
   main.py
```

① Version Python épinglée
② Configuration du projet
③ lockfile généré automatiquement

Pour uv, la référence des dépendances est le `uv.lock`. Ce fichier est à ajouter au référentiel. il décrit la version exacte des dépendances utilisés et sera utilisé pour le partage du projet.

### 20.6.1 Installer des dépendances

#### 20.6.1.1 Dépendances de production

```
uv add requests
uv add "httpx>=0.27"
uv add fastapi uvicorn
```

uv met à jour le `pyproject.toml` :

```
[project]
dependencies = [
    "httpx>=0.27",
    "fastapi>=0.110",
    "uvicorn>=0.29",
]
```

#### 20.6.1.2 Dépendances de développement

```
uv add --dev pytest
uv add --dev ruff mypy
uv add --dev "pytest-cov>=5.0"
```

uv crée la section dans le `pyproject.toml` :

```
[dependency-groups]
dev = [
    "pytest>=8.0",
    "ruff>=0.4",
    "mypy>=1.9",
    "pytest-cov>=5.0",
]
```

#### 20.6.1.3 Supprimer une dépendance

```
uv remove requests
```

### 20.6.2 Synchroniser l'environnement

Lors de la récupération d'un projet, on utiliser le fichier `uv.lock` pour configurer son environnement :

```
uv sync
uv sync --no-dev
```

`uv sync` crée ou met à jour `.venv/`. L'option `--no-dev` permet de n'installer que les dépendances de prod.

### 20.6.3 Lancer des commandes

Pas besoin d'activer le virtualenv :

```
uv run python main.py
uv run pytest
uv run ruff check .
uv run mypy src/
```

`uv run` détecte automatiquement le `.venv/` du projet.

### 20.6.4 Le fichier `uv.lock`

`uv.lock` est généré automatiquement. Il contient les versions **exactes** de toutes les dépendances (directes et transitives), garantissant la reproductibilité sur toutes les machines.

```
uv lock
```

**À committer dans Git.** Ne pas l'éditer manuellement.

### 20.6.5 Compatibilité

uv génère un `pyproject.toml` standard (PEP 621) : les projets restent compatibles avec `pip` et les autres outils de l'écosystème Python.

Si vous souhaitez revenir vers l'usage de `pip` classique, vous pouvez exporter les requirements :

```
# Exporter vers requirements.txt (CI, Docker…)
uv export --format requirements-txt > requirements.txt
uv export --format requirements-txt --no-dev > requirements.txt
```

### 20.6.6 Pour le packaging

Vous pouvez utiliser `uv` pour le packaging. Il faut indiquer le build-system dans le `pyproject.toml` :

```
[build-system]
requires = ["uv_build >= 0.11.7, <0.12.0"]
build-backend = "uv_build"
```

## 20.7 Références

— Documentation officielle uv
— GitHub astral-sh/uv
— Python Developer Tooling Handbook — uv

# 21 Créer un package

Pour créer un package, vous devez partir de la structure src-based et avoir un fichier `pyproject.toml` fonctionnel.

## 21.1 Utiliser `setuptools`

`setuptools` est l'outil historique. Une fois le fichier configuré, s'assurer de disposer des dépendances

```
pip install --upgrade setuptools[core]
pip install --upgrade build
```

Et lancer

```
python -m build
```

Ce qui créera un `tar.gz` et un `.whl` dans le répertoire `dist`.

## 21.2 Utiliser `uv`

uv veut rendre les choses plus faciles. Vous devez juste exécuter :

```
uv build
```